

NSS COLLEGE APPLICATION

Backend Implementation & Architecture Document

B.Tech College — National Service Scheme (NSS)

Document Item	Value
Project	NSS College Management Application
Scope	Backend architecture, implementation and operational design
Target	B.Tech / Engineering College
Recommended Runtime	Node.js with TypeScript
Recommended Framework	NestJS
Database	PostgreSQL
Cache / Queue	Redis where required
API	Versioned REST API
Authentication	Institution-approved authentication/session or token architecture
Document Version	1.0

Purpose: Define a production-oriented backend implementation for the NSS application, including domain architecture, APIs, database design, authentication, authorization, attendance integrity, reporting, background processing, observability, testing and deployment.

1. Backend Objectives

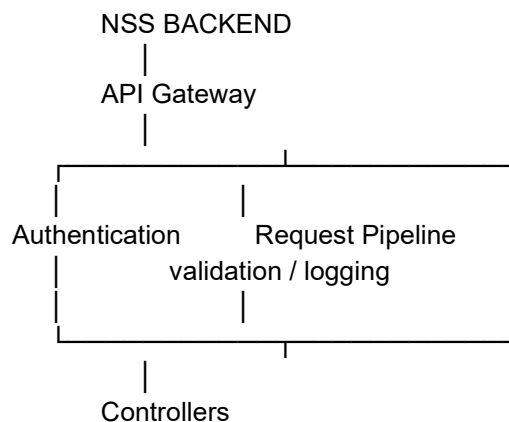
The backend is the authoritative application and security boundary. It owns identity, authorization, NSS business rules, data integrity, attendance validation, service-hour calculations, reports, audit records and institutional workflows.

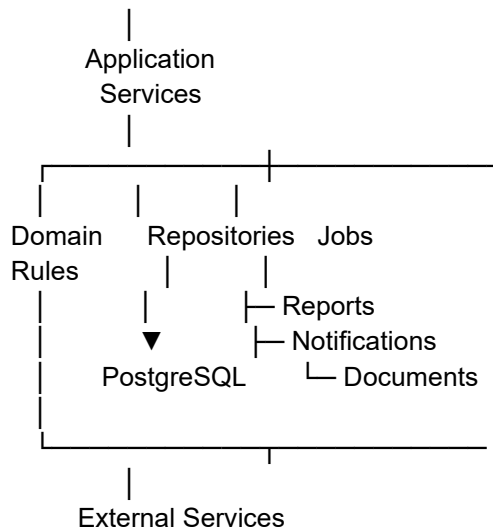
- Expose a stable, versioned API for the frontend.
- Enforce authentication and role/permission authorization server-side.
- Maintain transactional consistency for attendance, registration and service-hour records.
- Provide searchable and paginated access to institutional data.
- Generate reports and documents without blocking normal API requests.
- Maintain auditable records for sensitive administrative actions.
- Support reliable backups, recovery and operational monitoring.

2. Recommended Backend Technology Stack

Technology	Purpose	Implementation Guidance
Node.js + TypeScript	Application runtime	Strict TypeScript configuration; avoid implicit any.
NestJS	HTTP/application framework	Modules, controllers, services, guards, pipes and dependency injection.
PostgreSQL	Primary database	Relational integrity, transactions, constraints and indexes.
Redis	Cache/rate limits/queue support	Use only for workloads that benefit from it; PostgreSQL remains source of truth.
BullMQ or equivalent	Background jobs	Reports, notifications and other asynchronous work.
OpenAPI	API contract	Generate and maintain versioned API documentation.
Pino or equivalent	Structured logging	JSON logs with request/correlation IDs.
Docker	Packaging	Reproducible development and deployment.
CI/CD	Automation	Lint, type-check, test, migration verification and build gates.

3. Backend Architecture





Use a modular monolith initially. For a college NSS system, prematurely splitting into microservices adds operational complexity without a demonstrated scaling requirement. The modules should have clean boundaries so individual components can be extracted later if needed.

4. Domain Modules

Module	Responsibilities
Auth	Login, sessions/tokens, password/account lifecycle where applicable, current-user context.
Users	User identity, profile and account status.
Roles & Permissions	RBAC and permission checks.
Volunteers	Volunteer profile, academic information and NSS participation.
Units	NSS unit and Programme Officer relationships.
Events	Event lifecycle, eligibility, registration and capacity.
Attendance	Attendance sessions, check-ins, verification and corrections.
Service Hours	Authoritative verified-hour calculations and adjustments.
Announcements	Publishing, targeting and expiry.
Notifications	In-app/email notification orchestration as approved.
Achievements	Recognition records and approval.
Certificates	Eligibility, issuance and certificate records.
Reports	Queries, aggregations and asynchronous report generation.
Documents	Metadata and controlled media/document references.
Audit	Security-sensitive and administrative audit events.

5. Project Folder Structure

```

src/
├── main.ts
├── app.module.ts
├── config/
├── common/
├── guards/

```

- interceptors/
- filters/
- pipes/
- decorators/
- errors/
- logging/
- auth/
- users/
- roles/
- volunteers/
- units/
- events/
- attendance/
- service-hours/
- announcements/
- notifications/
- achievements/
- certificates/
- documents/
- reports/
- audit/
- database/
- jobs/

6. Database Architecture

PostgreSQL should be the system of record. Use foreign keys, unique constraints, check constraints and transactions to enforce invariants at the database boundary as well as in application code.

Core Table	Purpose
users	Authentication-linked identity and account state.
roles	Role definitions.
permissions	Fine-grained permissions if required.
user_roles	User-role assignments.
volunteers	NSS volunteer profile.
nss_units	NSS unit records.
unit_memberships	Volunteer-to-unit membership history.
events	NSS event records and lifecycle.
event_registrations	Volunteer event registration.
attendance_sessions	Attendance windows/sessions.
attendance_records	Individual attendance results.
attendance_corrections	Correction requests and decisions.
service_hour_entries	Authoritative verified service-hour ledger.
announcements	Published notices.
notifications	User notification records.
achievements	Approved achievement records.
certificates	Certificate issuance metadata.
documents	Uploaded document metadata.
reports	Report generation metadata/status.
audit_logs	Security and administrative audit events.

7. Core Data Integrity Rules

- A volunteer cannot be registered twice for the same event.
- An attendance record must reference a valid volunteer and attendance session.
- Attendance must be accepted only while the server-side session is valid.
- Official service hours must derive from verified attendance/approved activity records, not arbitrary client input.
- Event capacity must be enforced transactionally to prevent race-condition overbooking.
- Only authorized users may approve attendance corrections.
- Approved certificates and achievements must be immutable or require a controlled revision/audit workflow.
- Soft deletion/archive behavior should be used for institutional records where historical traceability is required.

8. Authentication

Choose the authentication mechanism based on college infrastructure. If institutional SSO exists, integrate with the approved identity provider. Otherwise use a secure application authentication flow.

1. Validate credentials using a dedicated authentication service.
2. Establish a secure session or short-lived access-token strategy.
3. Use refresh-token rotation if refresh tokens are used.
4. Never store plaintext passwords.
5. Rate-limit authentication attempts.
6. Return generic authentication failures to avoid account enumeration.
7. Record security-relevant authentication events.

9. Authorization & RBAC

Role	Typical Permissions
Volunteer	View own profile/events/attendance/hours; register for eligible events; submit permitted requests.
Student Leader	Limited unit/event operational actions according to policy.
Programme Officer	Manage assigned unit events, attendance, reports and volunteer workflows.
Coordinator	Broader NSS management and reporting.
Administrator	Institution-level configuration, users, roles, units and audit access.

Implement authorization at the service/API boundary. Controller-level guards can provide early rejection, but critical permissions should also be enforced in application services so business operations cannot bypass authorization through alternate code paths.

10. REST API Standards

- Version public APIs, for example /api/v1.
- Use resource-oriented endpoints.
- Use consistent pagination, sorting and filtering parameters.
- Return stable structured error responses.
- Validate all request bodies, path parameters and query parameters.
- Use appropriate HTTP status codes.
- Do not expose database entities directly as API contracts.

- Use DTOs/response models so internal schema changes do not automatically become API breaking changes.

Operation	Example Endpoint
Login	POST /api/v1/auth/login
Current user	GET /api/v1/auth/me
Events	GET /api/v1/events
Event details	GET /api/v1/events/:eventId
Create event	POST /api/v1/events
Register	POST /api/v1/events/:eventId/registrations
Attendance session	POST /api/v1/events/:eventId/attendance-sessions
Check-in	POST /api/v1/attendance/check-ins
Attendance	GET /api/v1/events/:eventId/attendance
Service hours	GET /api/v1/volunteers/:volunteerId/service-hours
Announcements	GET /api/v1/announcements
Reports	POST /api/v1/reports

11. Structured Error Contract

```
{
  "error": {
    "code": "EVENT_REGISTRATION_CLOSED",
    "message": "Registration is closed for this event.",
    "requestId": "correlation-id"
  }
}
```

Do not return stack traces, SQL errors, secret values or internal infrastructure details to clients. Keep detailed diagnostics in protected server logs.

12. Event Lifecycle

```
Draft
↓
Published
↓
Registration Open
↓
Registration Closed
↓
Ongoing
↓
Completed
↓
Archived
```

Cancellation may be allowed from approved intermediate states.

- Validate event date/time and registration windows.
- Validate NSS unit ownership/permission.
- Enforce capacity safely under concurrency.

- Prevent registration after closure.
- Prevent invalid lifecycle transitions.

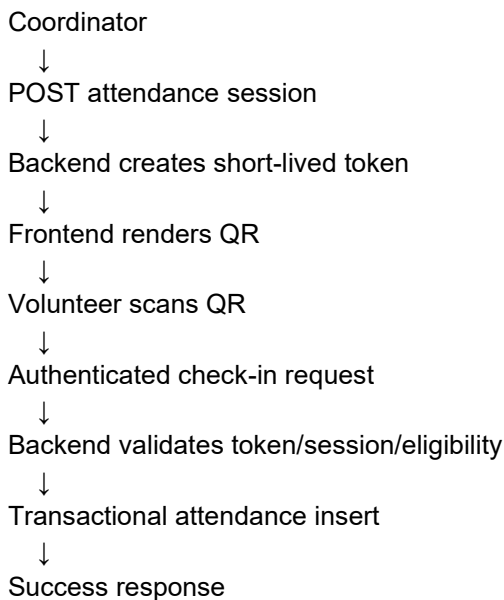
13. Attendance Backend Design

Attendance is a high-integrity domain. The backend must decide whether a check-in is valid.

13.1 Attendance Session

- Create a session tied to an event.
- Generate a short-lived, non-predictable attendance token if QR is used.
- Store session validity window and state server-side.
- Bind check-in to the authenticated volunteer identity.
- Reject expired, reused, invalid or unauthorized attendance requests.

13.2 QR Flow



13.3 Concurrency

Use database constraints and transactions to prevent duplicate attendance and registration races. Never rely only on an application-level 'check then insert' sequence.

14. Service Hours Backend

Treat service hours as a derived/verified institutional record. Prefer a ledger model so every adjustment has provenance.

Entry Field	Purpose
volunteer_id	Owner of the hour entry.
event_id	Source event where applicable.
hours	Verified quantity.
source_type	Attendance/activity/approved adjustment.
status	Pending/verified/rejected according to workflow.
approved_by	Authorized approver.
approved_at	Approval timestamp.

reason	Required for manual adjustment.
--------	---------------------------------

The frontend should request totals from the backend. Do not accept a client-supplied total as the official service-hour value.

15. Announcements & Notifications

- Create announcements with audience targeting.
- Publish and expire announcements according to server time.
- Generate notifications from events and workflow actions where configured.
- Use an asynchronous queue for high-volume notification delivery.
- Persist notification status where delivery tracking is required.

16. File & Document Handling

- Do not store arbitrary uploads directly in the application container filesystem for production.
- Use approved object storage or managed document storage.
- Validate file size, MIME type and extension server-side.
- Generate non-guessable object identifiers.
- Do not trust the client-provided filename or MIME type.
- Scan files where institutional security policy requires it.
- Store metadata and ownership references in PostgreSQL.
- Authorize every document download.

17. Reporting & Background Jobs

Large reports should not block normal HTTP requests. Create a report job, process it asynchronously, store status/metadata, and provide a secure download endpoint when ready.

POST /reports

↓

Create report record: QUEUED

↓

Queue job

↓

Worker generates report

↓

Report record: READY / FAILED

↓

Authorized download

- Report filters must be validated and scoped to the user's permissions.
- Use streaming or bounded-memory generation for large exports.
- Expire temporary report files according to retention policy.
- Record who requested sensitive reports.

18. Audit Logging

Audit logs should capture security-sensitive and administrative actions without storing secrets.

Event	Example Data
Login success/failure	User reference, timestamp, outcome, request metadata.
Role change	Actor, target user, previous/new role.
Event modification	Actor, event, action.
Attendance correction	Requester, approver, reason, old/new state.
Service-hour adjustment	Actor, volunteer, quantity, reason.
Certificate issuance	Issuer, volunteer, certificate reference.
Administrative export	Actor, report type, filter scope.

19. Backend Security Requirements

- Apply authentication to all protected endpoints.
- Apply authorization to every sensitive operation.
- Validate and constrain all external input.
- Use parameterized queries/ORM query mechanisms.
- Protect against mass assignment by explicit DTO mapping.
- Rate-limit login and other abuse-sensitive endpoints.
- Configure CORS to approved origins.
- Use security headers appropriate to the deployment architecture.
- Avoid verbose production error responses.
- Rotate credentials and secrets through a secure secret-management mechanism.
- Encrypt data in transit; use encrypted database/storage services where required.
- Back up the database and test restoration.
- Keep dependencies patched and run automated vulnerability checks.
- Use least-privilege database and infrastructure accounts.

20. Validation & Business Rules

Use three layers where appropriate: transport validation, application/domain validation and database constraints.

Layer	Example
DTO / transport	UUID format, string length, required fields.
Application/domain	End time after start; event registration eligibility.
Database	Unique registration; foreign keys; non-negative hours.

21. Transaction Boundaries

- Event registration with capacity enforcement.
- Attendance insertion with duplicate protection.
- Attendance correction approval plus corresponding service-hour adjustment.
- Certificate issuance with eligibility state update.
- Role/permission changes plus audit record where atomicity is required.

Transactions should be short and deliberate. Avoid holding database transactions open while calling external services.

22. Caching & Rate Limiting

- Cache only data that tolerates controlled staleness.
- Never cache authorization decisions longer than policy permits.
- Use cache invalidation after mutations where needed.
- Use Redis for distributed rate limiting when multiple backend instances are deployed.
- Do not use Redis as the authoritative attendance or service-hour store.

23. Observability

- Structured JSON logs.
- Request/correlation IDs.
- HTTP latency metrics.
- Database query latency monitoring.
- Background job success/failure metrics.
- Authentication failure monitoring.
- Error-rate monitoring.
- Health and readiness endpoints.
- Alerting for repeated failures or resource exhaustion.

24. Health Checks

Endpoint/Check	Purpose
/health/live	Process is running.
/health/ready	Application can serve traffic; dependencies meet readiness requirements.
Database connectivity	Verify database availability.
Queue connectivity	Verify worker infrastructure where used.

25. Backend Testing Strategy

Test Level	Scope
Unit	Domain services, validators, authorization policies and calculation logic.
Integration	Repositories, database constraints and transactional workflows.
API	Controllers, DTO validation, status codes and response contracts.
E2E	Complete volunteer/coordinator/admin workflows.
Security	Authorization bypass attempts, rate limits, input validation and file controls.
Load	Registration, attendance and reporting under expected peak load.

25.1 Boundary-Value Tests

- Registration at exactly the participant capacity.
- Registration one participant beyond capacity.
- Event start/end times equal or reversed.
- Attendance exactly at session expiry.
- Duplicate attendance submission.
- Zero, negative and excessive service-hour values.
- Maximum allowed title/description length and one character beyond it.
- Empty and maximum-size report filters.
- Expired authentication/session.
- Unauthorized access to another volunteer's records.

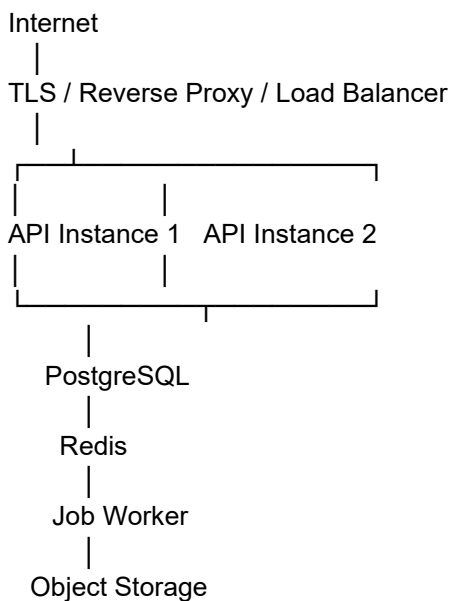
26. API Documentation

- Generate OpenAPI documentation from typed controller DTOs.
- Document authentication requirements.
- Document request/response schemas.
- Document common errors and authorization failures.
- Version breaking API changes.
- Provide examples for frontend integration.

27. Database Migrations

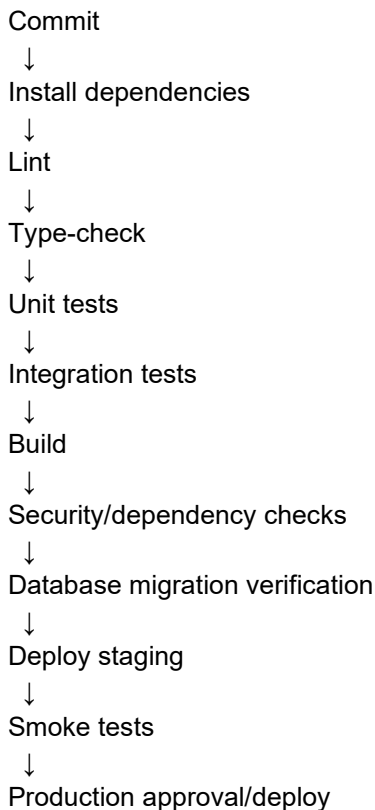
- All schema changes must be version-controlled migrations.
- Never rely on manually edited production schemas.
- Test migrations against a clean database and a representative existing database.
- Include rollback strategy where practical.
- Back up before high-risk production migrations.

28. Deployment Architecture



- Run multiple API instances when availability requirements justify it.
- Keep application instances stateless where possible.
- Use a managed PostgreSQL service or hardened database deployment.
- Run background workers separately from request-serving processes.
- Use object storage for documents.
- Store secrets outside source control.

29. CI/CD Pipeline



30. Step-by-Step Backend Implementation Roadmap

Phase 1 — Foundation

- Create NestJS TypeScript project
- Configuration module
- Database connection
- Global validation
- Global exception filter
- Structured logging
- Health endpoints

Phase 2 — Identity

- Users
- Authentication
- Session/token lifecycle

- Roles and permissions
- Authorization guards/policies
- Audit events

Phase 3 — NSS Core

- Volunteers
- NSS units
- Memberships
- Events
- Registration
- Event lifecycle

Phase 4 — Attendance

- Attendance sessions
- Short-lived QR/token flow
- Check-in validation
- Duplicate protection
- Correction workflow

Phase 5 — Institutional Records

- Service-hour ledger
- Announcements
- Notifications
- Achievements
- Certificates
- Document metadata

Phase 6 — Reporting

- Report queries
- Aggregations
- Background workers
- CSV/PDF generation
- Secure downloads

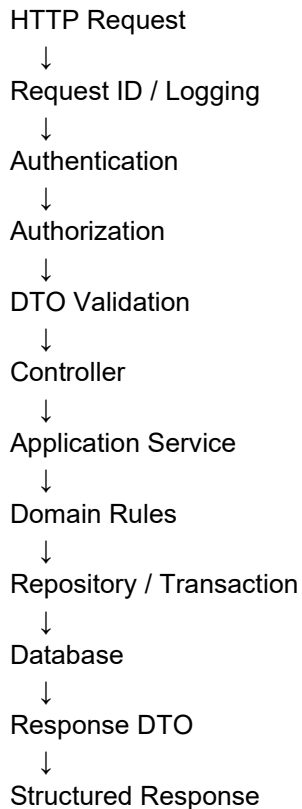
Phase 7 — Hardening

- Rate limiting
- Security headers
- Dependency scanning
- Load testing
- Backup/restore validation
- Observability

Phase 8 — Production

- CI/CD
- Staging verification
- Migration deployment
- Production deployment
- Smoke tests
- Operational runbook

31. Standard Request Lifecycle



32. Backend-to-SRS Traceability

SRS Area	Backend Implementation
Authentication & Users	Auth, users, roles, sessions/tokens, audit.
Volunteer Management	Volunteer and membership modules with validation and pagination.
NSS Units	Unit management, officer assignments and authorization scope.
Events	Event lifecycle, registration, capacity and eligibility.
Attendance	Sessions, QR/check-in validation, correction workflow.
Service Hours	Verified service-hour ledger and calculation queries.
Reports	Filtered queries, asynchronous generation and secure downloads.
Announcements	Publishing, targeting, expiry and notification triggers.
Achievements	Approval-controlled achievement records.
Certificates	Eligibility, issuance metadata and secure access.
Administration	RBAC, audit logs, configuration and institution-level controls.

33. Backend Definition of Done

- All protected endpoints require authentication.
- All sensitive operations enforce server-side authorization.
- All external input is validated.
- Critical business invariants have database constraints where applicable.

- Transactions cover race-sensitive operations.
- API responses use stable DTOs and structured errors.
- Audit logging covers sensitive administrative changes.
- Automated unit, integration and E2E tests cover critical workflows.
- Database migrations are version controlled.
- Production secrets are managed outside source control.
- Backups and restore procedures have been tested.
- Health checks and structured logs are available.
- The production build and deployment pipeline are reproducible.

34. Engineering Trade-Off Summary

Memory: A modular monolith with PostgreSQL as source of truth and bounded caching keeps memory usage predictable; asynchronous workers isolate heavy report generation.

Latency: Indexed queries, connection pooling, short transactions and selective Redis caching keep normal API paths fast; background jobs prevent expensive reports/notifications from blocking requests.

Operational Overhead: A modular monolith is substantially simpler to operate than microservices while preserving domain boundaries; Redis, workers, object storage, observability and CI/CD add infrastructure cost but are justified as usage grows.